Chủ đề 5: Lưu trữ dữ liệu dài hạn

Nội dung chính:

- 5.1. Lưu trữ dữ liệu dạng khóa - giá trị trên Android.
- 5.2. Tạo cơ sở dữ liệu quan hệ cho ứng dụng trên Android.
- 5.3. Quản lý dữ liệu với thư viện Room.

Mục tiêu:

- Hiểu các phương pháp lưu trữ dữ liệu dài hạn trên Android.
- Biết cách sử dụng SharedPreferences và SQLite.
- Vận dụng thư viện Room để xây dựng ứng dụng.

Ứng dụng thực tế:

- Lưu trữ thông tin người dùng (tên, email, cấu hình).

Tổng quan về lưu trữ dữ liệu trên Android

- Các phương pháp lưu trữ:

- Lưu trữ khóa - giá trị với SharedPreferences.
- Cơ sở dữ liệu SQLite.
- Quản lý dữ liệu với Room.

Ưu điểm:

- Tích hợp sẵn trên Android.
- Hiệu quả và dễ triển khai.

Hạn chế:

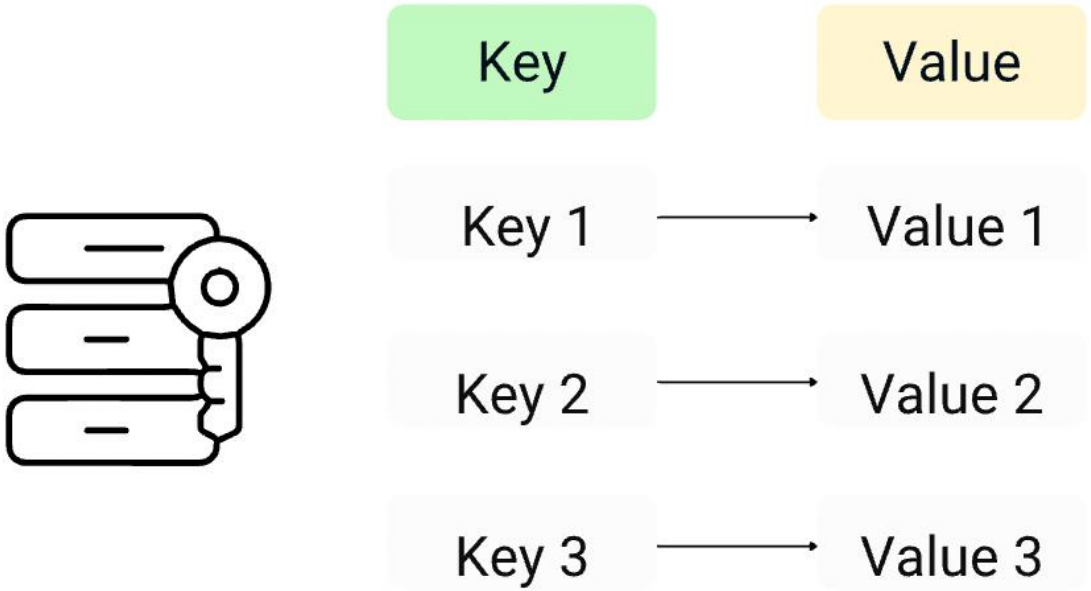
- Phương pháp không phù hợp với mọi loại dữ liệu.

SharedPreferences là gì?

- Một cách lưu trữ dữ liệu khóa-giá trị dạng XML.Dùng để lưu dữ liệu cấu hình hoặc thông tin không lớn.

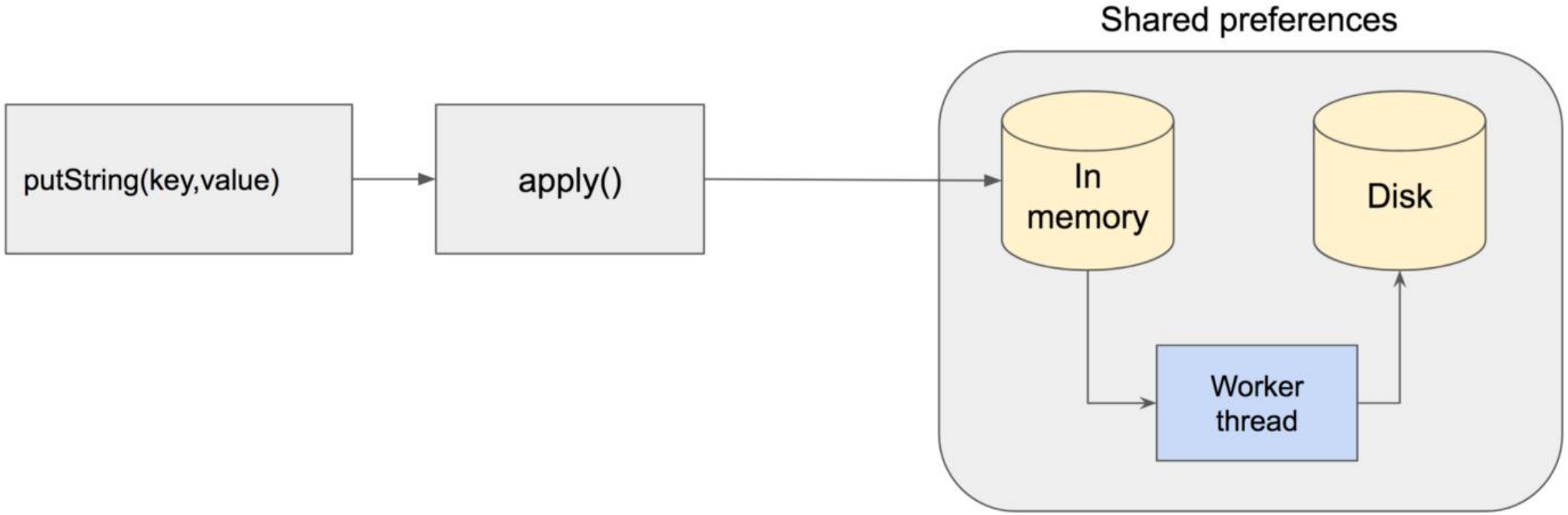
Lưu ý: Không lưu trữ dữ liệu phức tạp.

File lưu tại: /data/data/[package_name]/shared_prefs.



SharedPreferences là gì?

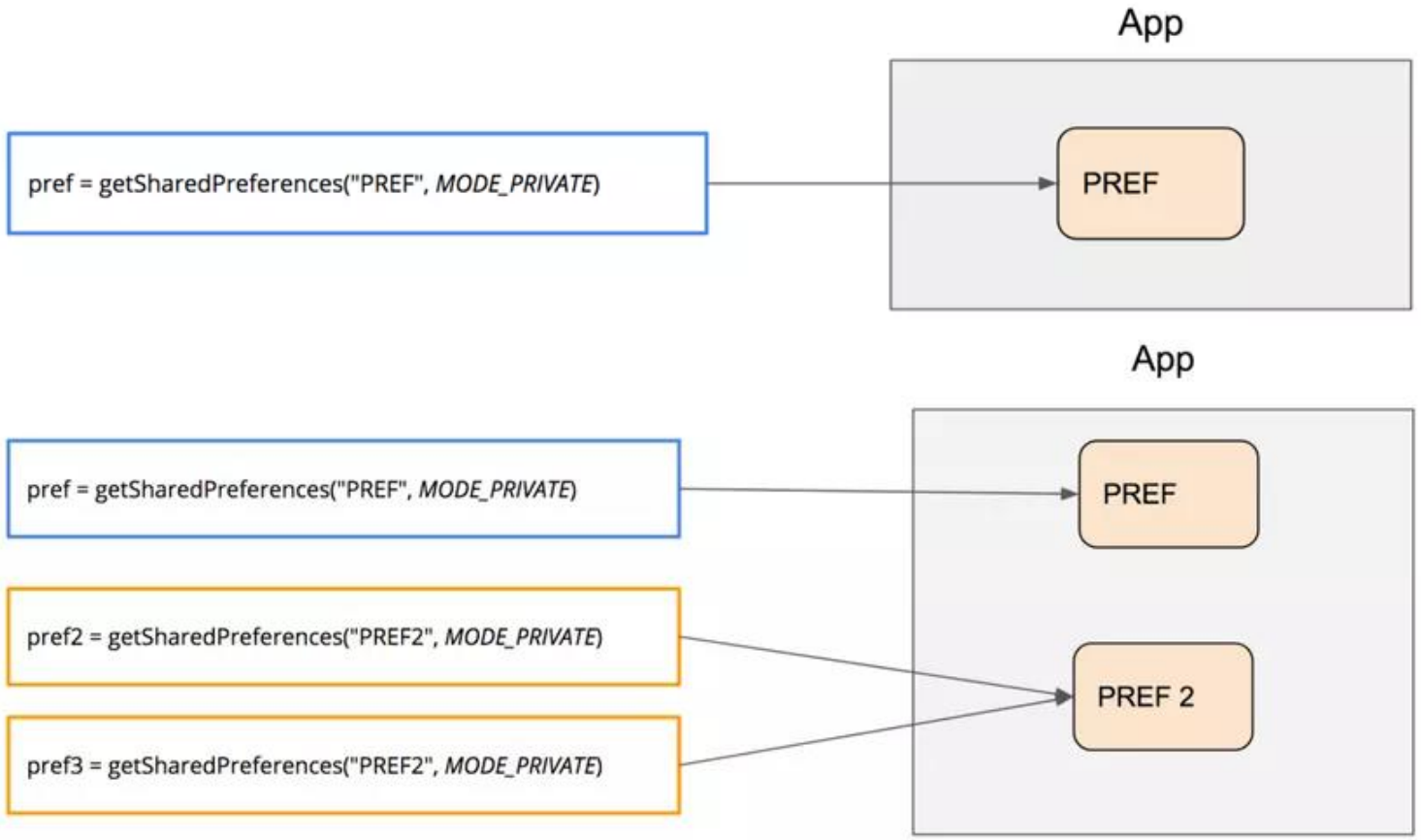
- Cơ chế hoạt động của SharedPreferences



Lấy đối tượng SharedPreferences:

1. Lấy đối tượng SharedPreferences:

val shPre = this.getSharedPreferences("file_name", Context.MODE_PRIVATE)



Ghi dữ liệu vào SharedPreferences

2. Tạo đối tượng Editor để chỉnh sửa:

```
val editor = shPre.edit()

editor.putString("key", "value")

editor.putInt("age", 30)

editor.apply()
```

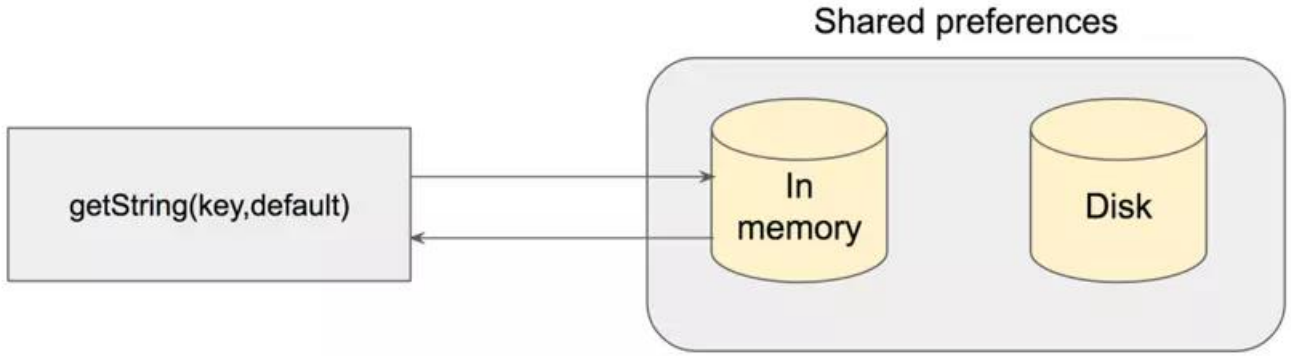
Đọc dữ liệu từ SharedPreferences

1. Lấy đối tượng SharedPreferences:

```
val shPre = this.getSharedPreferences("file_name", Context.MODE_PRIVATE)
```

2. Đọc giá trị:

```
val username = shPre.getString("username", "Unknown")
val age = shPre.getInt("age", -1)
println("Username: $username, Age: $age")
```



Xóa dữ liệu trong SharedPreferences

- Xóa một cặp key-value:

```
editor.remove("key")
editor.apply()
```

- Xóa toàn bộ:

```
editor.clear()
editor.apply()
```

Ví dụ: Sử dụng SharedPreferences

- Tạo giao diện với các nút Ghi, Đọc, Xóa.

- Lưu tên và email người dùng vào file myprefs.xml.

Code chính:

```
buttonSave.setOnClickListener {
    editor.putString("Name", name)
    editor.putString("Email", email)
    editor.apply()
}
```

Thực hành: SharedPreferences

Viết ứng dụng lưu thông tin người dùng:

- Nhập tên và email.
- Lưu dữ liệu khi nhấn nút Ghi.
- Hiển thị dữ liệu khi nhấn nút Đọc.
- Xóa dữ liệu khi nhấn nút Xóa.

Ưu và nhược điểm của SharedPreferences

- Ưu điểm:

- Dễ sử dụng.
- Phù hợp cho dữ liệu cấu hình nhỏ.

- Nhược điểm:

- Không phù hợp cho dữ liệu lớn hoặc phức tạp.
- Chỉ lưu được dưới dạng khóa-giá trị.

Tổng kết phần SharedPreferences

- SharedPreferences dùng để lưu trữ dữ liệu đơn giản.
- Lý tưởng cho lưu trữ cấu hình ứng dụng.
- Thực hành tạo ứng dụng để áp dụng kiến thức.

Tổng kết phần SharedPreferences

- SharedPreferences dùng để lưu trữ dữ liệu đơn giản.
- Lý tưởng cho lưu trữ cấu hình ứng dụng.
- Thực hành tạo ứng dụng để áp dụng kiến thức.

SQLite là gì?

- SQLite là cơ sở dữ liệu quan hệ tích hợp trong Android.

- Không cần server, dữ liệu lưu trữ trong file .db trên thiết bị.

Ưu điểm:

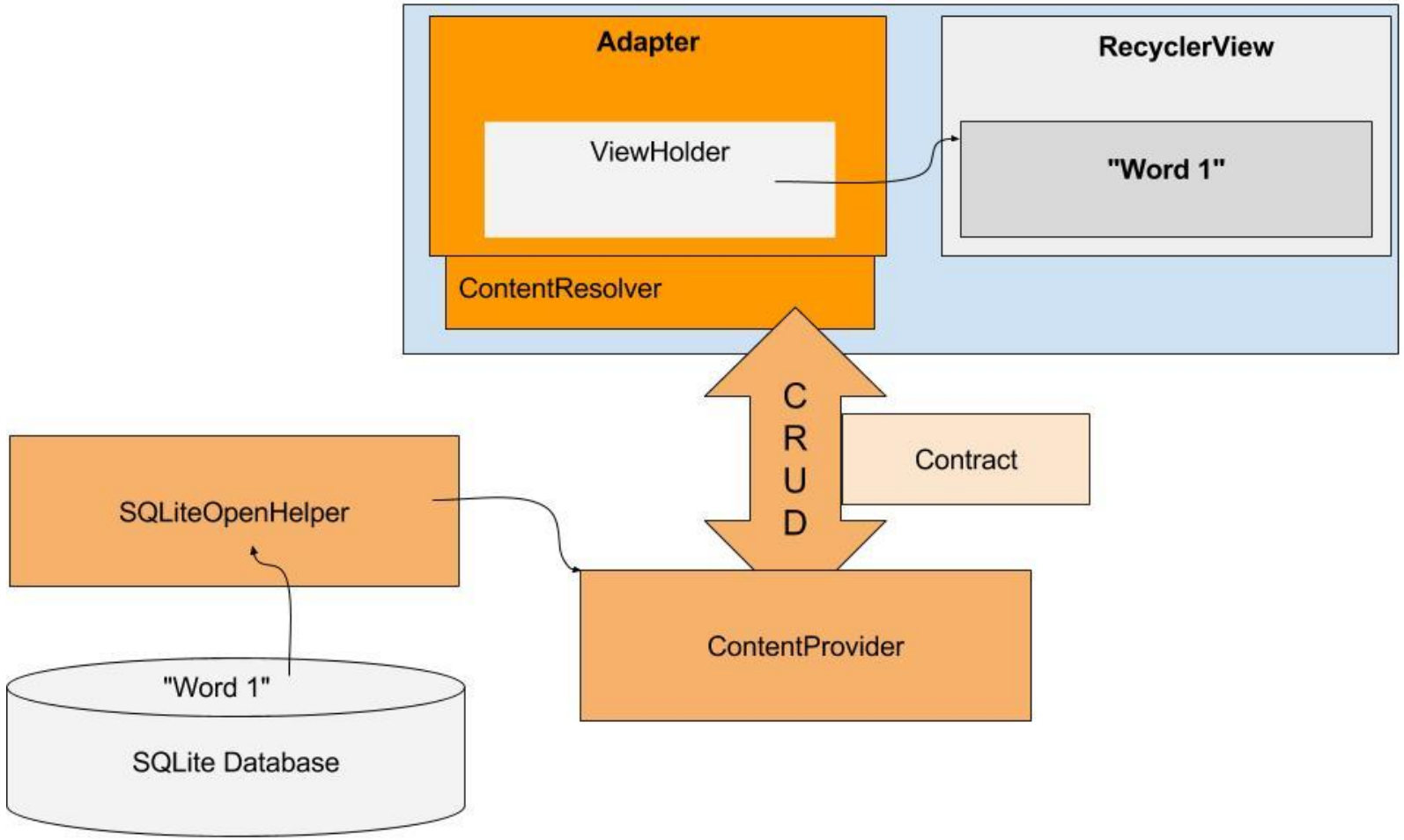
- Dễ sử dụng, hỗ trợ truy vấn SQL đầy đủ.

- Tương thích với Room Library.

Ví dụ:

```
val db = openOrCreateDatabase("MyDatabase.db", Context.MODE_PRIVATE, null)
```

SQLite là gì?


Adapter
ViewHolder
ContentResolver
RecyclerView
"Word 1"
C
R
U
D
Contract
SQLiteOpenHelper
ContentProvider
"Word 1"
SQLite Database

SQLite và các thành phần

SQLiteOpenHelper – Hỗ trợ quản lý cơ sở dữ liệuLớp SQLiteOpenHelper giúp tạo, cập nhật và quản lý cơ sở dữ liệu SQLite trong Android.

```
class MyDatabaseHelper(context: Context) : SQLiteOpenHelper(context, "MyDatabase.db", null, 1) {
    override fun onCreate(db: SQLiteDatabase) {
        // Tạo bảng khi cơ sở dữ liệu được khởi tạo
        db.execSQL("CREATE TABLE Users (id INTEGER PRIMARY KEY, name TEXT, age INTEGER)")
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        // Xử lý nâng cấp cơ sở dữ liệu
        db.execSQL("DROP TABLE IF EXISTS Users")
        onCreate(db)
    }
}
```

version trong SQLite

Cách hoạt động của version trong SQLite

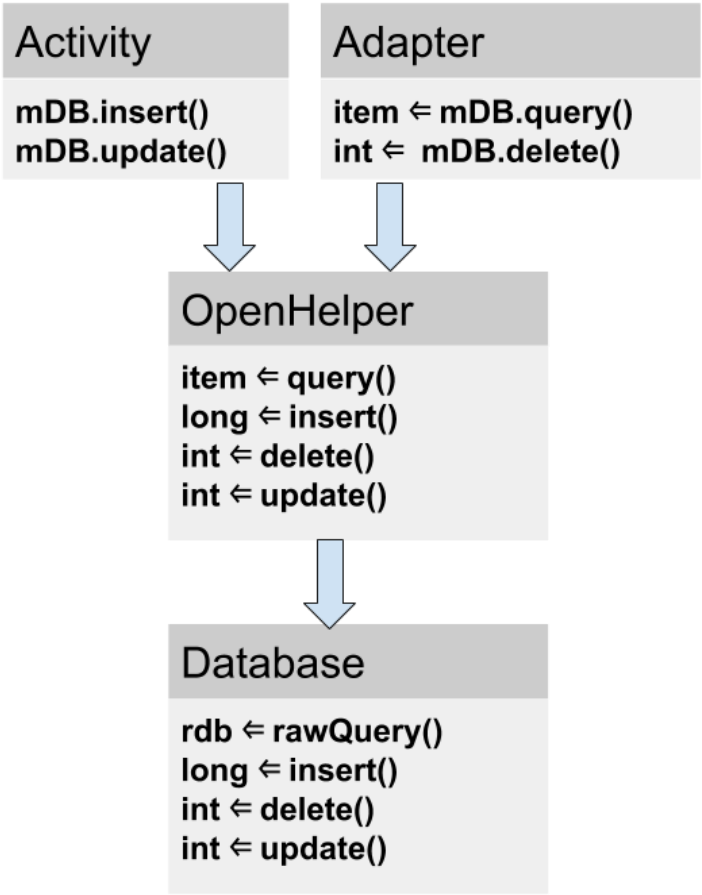
Phiên bản cơ sở dữ liệu được xác định bởi tham số version trong SQLiteOpenHelper.

Khi version thay đổi, phương thức onUpgrade() sẽ được gọi để cập nhật cơ sở dữ liệu.

Nếu cần hạ cấp cơ sở dữ liệu, có thể sử dụng onDowngrade().

```
class MyDatabaseHelper(context: Context) : SQLiteOpenHelper(context, "MyDatabase.db", null, 2) {
    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL("CREATE TABLE Users (id INTEGER PRIMARY KEY, name TEXT, age INTEGER)")
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        if (oldVersion < 2) {
            db.execSQL("ALTER TABLE Users ADD COLUMN email TEXT")
        }
    }
}
```



SQLiteOpenHelper

- SQLiteOpenHelper giúp quản lý việc tạo, nâng cấp cơ sở dữ liệu.

```
class MyDatabaseHelper(context: Context) : SQLiteOpenHelper(context, DATABASE_NAME, null, DATABASE_VERSION) {

    override fun onCreate(db: SQLiteDatabase) {
        val createTableQuery = """
            CREATE TABLE $TABLE_USERS (
                $COLUMN_ID INTEGER PRIMARY KEY,
                $COLUMN_NAME TEXT,
                $COLUMN_AGE INTEGER
            )
        """.trimIndent()
        db.execSQL(createTableQuery)
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        db.execSQL("DROP TABLE IF EXISTS $TABLE_USERS")
        onCreate(db)
    }

    companion object {
        private const val DATABASE_NAME = "MyDatabase.db"
        private const val DATABASE_VERSION = 1
        private const val TABLE_USERS = "Users"
        private const val COLUMN_ID = "ID"
        private const val COLUMN_NAME = "Name"
        private const val COLUMN_AGE = "Age"
    }
}
```

SQLiteDatabase

SQLiteDatabase – Thao tác với cơ sở dữ liệuSQLiteDatabase giúp thực hiện các thao tác như thêm, sửa, xóa, truy vấn dữ liệu.

```
val dbHelper = MyDatabaseHelper(context)
val db = dbHelper.writableDatabase

val values = ContentValues().apply {
    put("name", "Nam")
    put("age", 22)
}
db.insert("Users", null, values)
```

Hướng dẫn - Tạo cơ sở dữ liệu SQLite

- Tạo cơ sở dữ liệu và bảng dữ liệu.

```
val db = openOrCreateDatabase("MyDatabase.db", Context.MODE_PRIVATE, null)
db.execSQL("CREATE TABLE IF NOT EXISTS Users (ID INTEGER PRIMARY KEY, Name TEXT, Age INTEGER)")
```

Dữ liệu lưu tại: /data/data/[package_name]/databases

Hướng dẫn - Thêm dữ liệu vào SQLite

- Chèn dữ liệu bằng câu lệnh SQL.

```
val insertSQL = "INSERT INTO Users (Name, Age) VALUES (?, ?)"
val stmt = db.compileStatement(insertSQL)
stmt.bindString(1, "John")
stmt.bindLong(2, 30)
stmt.executeInsert()
```

Hướng dẫn - Đọc dữ liệu từ SQLite

- Truy vấn và đọc dữ liệu bằng cursor.

```
val querySQL = "SELECT * FROM Users"
val cursor = db.rawQuery(querySQL, null)
while (cursor.moveToNext()) {
    val name = cursor.getString(cursor.getColumnIndex("Name"))
    val age = cursor.getInt(cursor.getColumnIndex("Age"))
    println("User: $name, Age: $age")
}
cursor.close()
```

Hướng dẫn - Cập nhật dữ liệu trong SQLite

- Cập nhật dữ liệu bằng câu lệnh SQL.

```
val updateSQL = "UPDATE Users SET Age = ? WHERE Name = ?"
val stmt = db.compileStatement(updateSQL)
stmt.bindLong(1, 35)
stmt.bindString(2, "John")
stmt.executeUpdateDelete()
```

Hướng dẫn - Xóa dữ liệu trong SQLite

- Xóa dữ liệu bằng câu lệnh SQL.

```
val deleteSQL = "DELETE FROM Users WHERE Name = ?"
val stmt = db.compileStatement(deleteSQL)
stmt.bindString(1, "John")
stmt.executeUpdateDelete()
```

Thực hành: SQLite

- Viết ứng dụng quản lý danh sách người dùng:
  - Thêm, sửa, xóa người dùng.
  - Hiển thị danh sách người dùng.

Ưu và nhược điểm của SQLite

- Ưu điểm:

- Truy vấn SQL đầy đủ.
- Tích hợp sẵn trong Android.

- Nhược điểm:

- Không tự động ánh xạ dữ liệu với đối tượng.
- Cần viết nhiều mã cho các thao tác.

Tổng kết phần SQLite

- SQLite là cơ sở dữ liệu quan hệ nhẹ, phù hợp lưu trữ dữ liệu cục bộ.
- Cần kết hợp với SQLiteOpenHelper để quản lý hiệu quả.
- Thực hành tạo ứng dụng để áp dụng kiến thức.

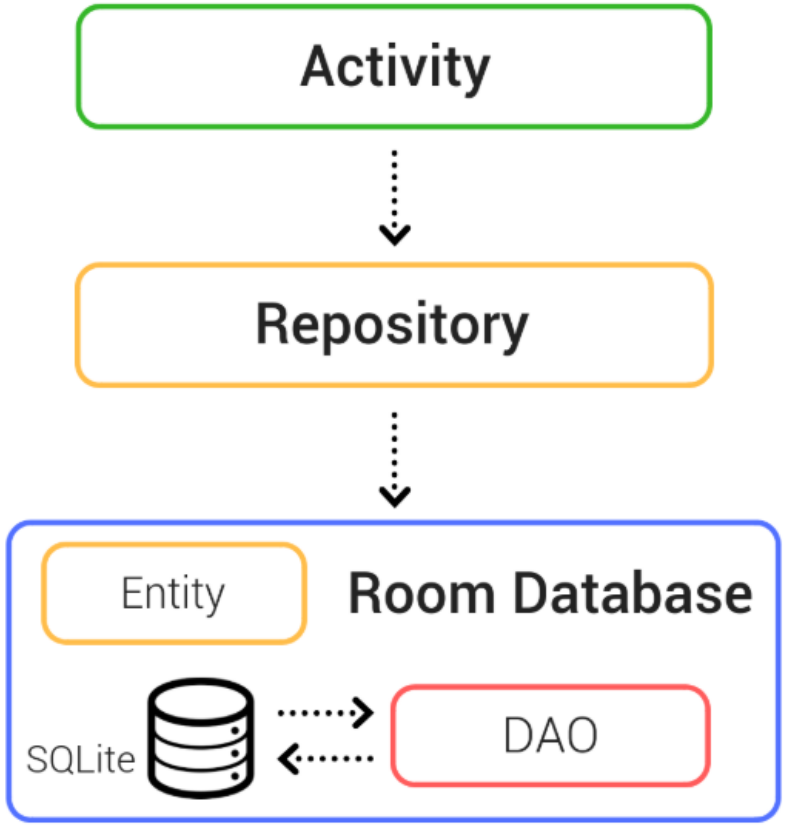
Room Library là gì?

- Room là một thư viện ORM (Object-Relational Mapping) của Android.
- Quản lý dữ liệu dựa trên SQLite nhưng dễ sử dụng hơn.

Ưu điểm:

- Tự động ánh xạ dữ liệu giữa đối tượng và bảng.
- Tích hợp tốt với LiveData và ViewModel.

```
@Entity
data class User(
    @PrimaryKey val id: Int,
    val name: String,
    val age: Int
)
```



Cấu trúc Room

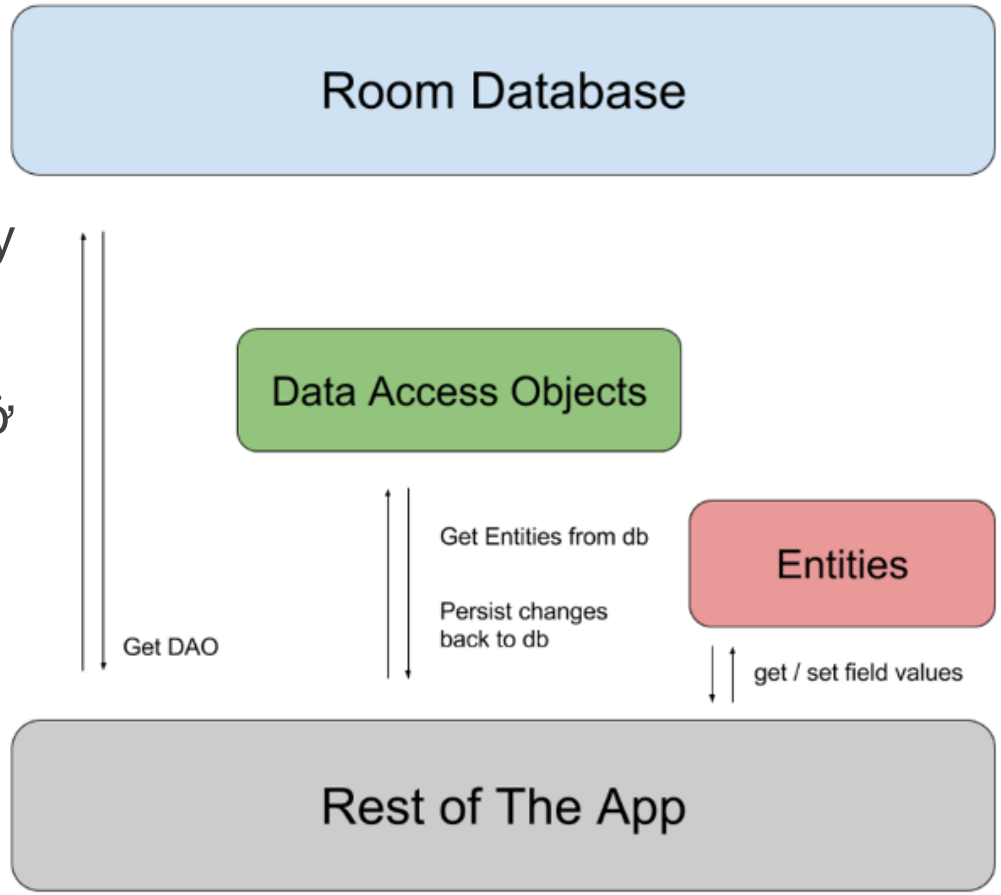
Cấu trúc Room gồm 3 thành phần chính:

- Entity: Biểu diễn bảng dữ liệu.

- DAO (Data Access Object): Quản lý truy vấn.

- Database: Điểm truy cập chính của cơ sở dữ liệu.

Entity -> DAO -> Database.

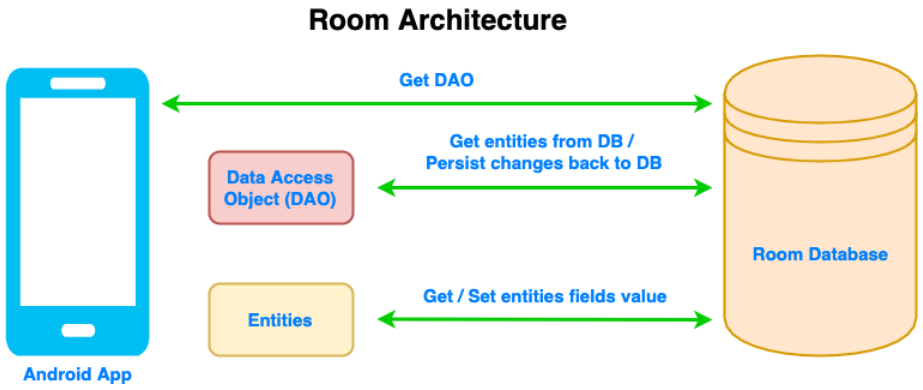


Tạo Entity trong Room

- Entity biểu diễn bảng dữ liệu trong Room.

Ví dụ:

```
@Entity(tableName = "users")
data class User(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val name: String,
    val age: Int
)
```



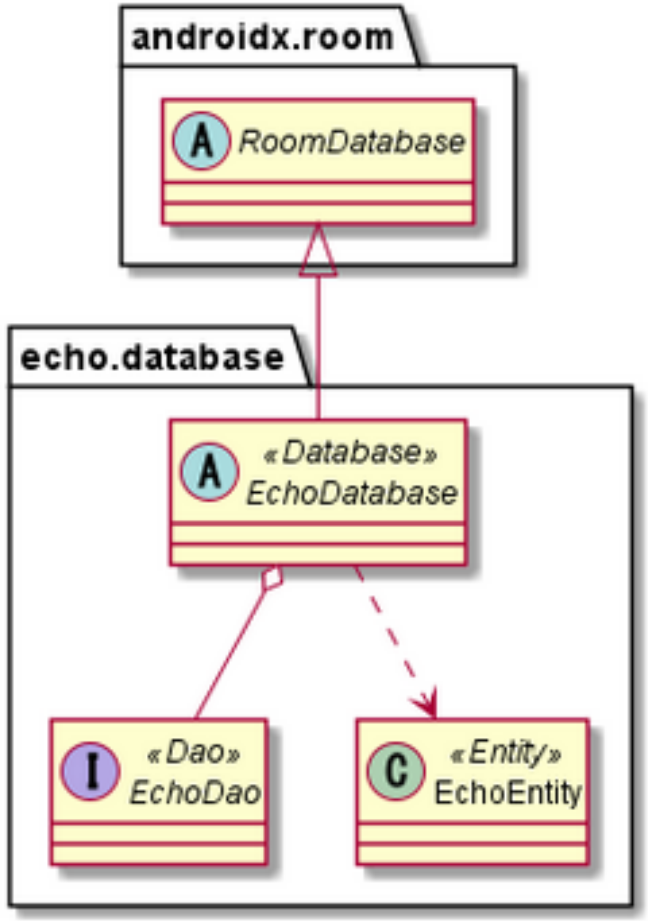
Tạo DAO

- DAO (Data Access Object) quản lý các truy vấn dữ liệu.

Ví dụ:

```
@Dao
interface UserDao {
    @Insert
    fun insertUser(user: User)

    @Query("SELECT * FROM users")
    fun getAllUsers(): List<User>
}
```

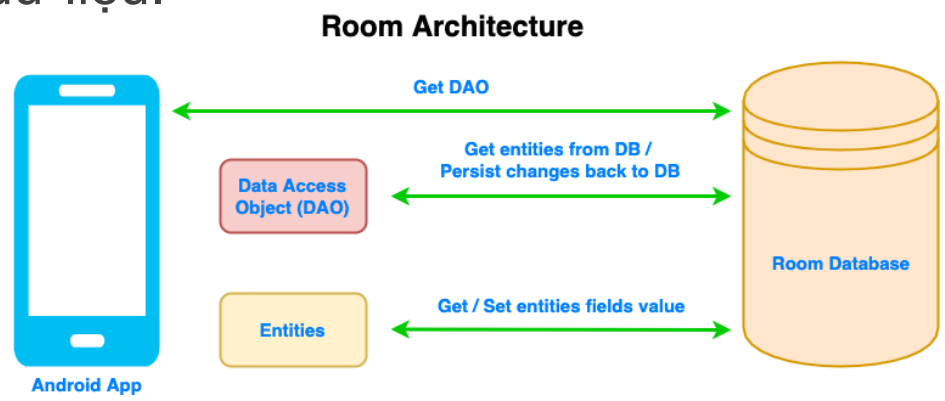


Tạo Database

- Database là điểm truy cập chính để thao tác dữ liệu.

Ví dụ:

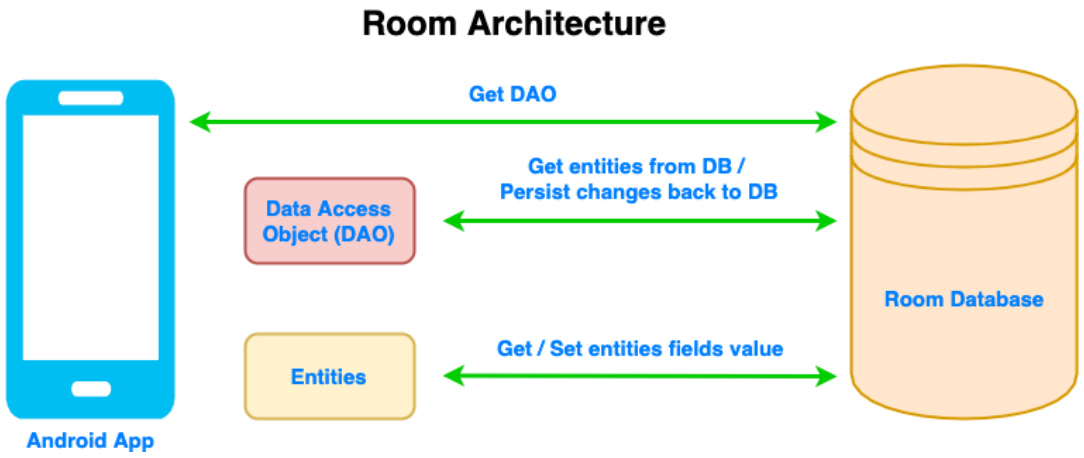
```
@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
```



Khởi tạo Room Database

- Khởi tạo hoặc mở cơ sở dữ liệu Room:

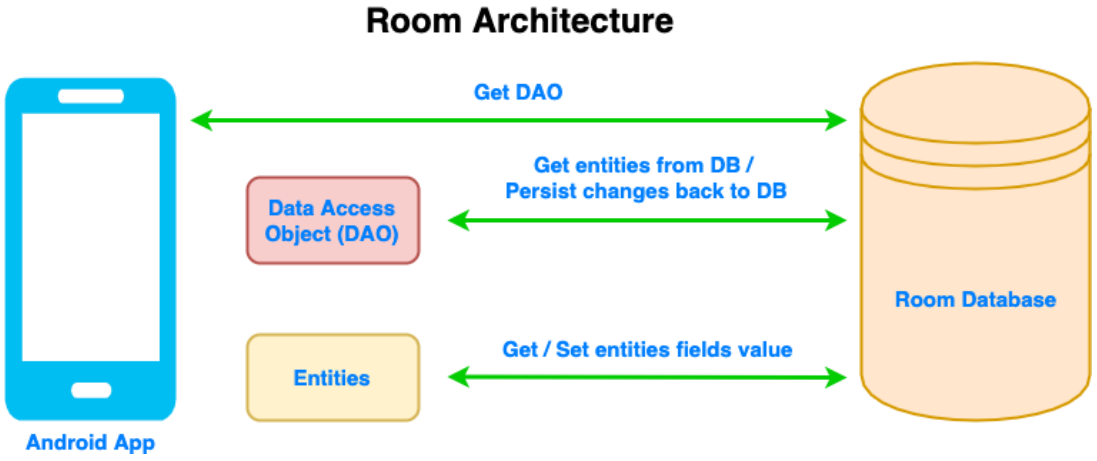
```
val db = Room.databaseBuilder(
    applicationContext,
    AppDatabase::class.java,
    "app_database"
).build()
```



Thực hiện các thao tác với Room

- Sử dụng DAO để thao tác dữ liệu.

```
val userDao = db.userDao()
userDao.insertUser(User("Alice", 25))
val users = userDao.getAllUsers()
println(users)
```



Tích hợp LiveData với Room

- Tích hợp LiveData giúp tự động cập nhật UI khi dữ liệu thay đổi.

Ví dụ:

```
@Query("SELECT * FROM users")
fun getAllUsersLive(): LiveData<List<User>>
```

Ưu và nhược điểm của Room

- Ưu điểm:

- Tự động hóa nhiều thao tác.
- Giảm lỗi SQL thủ công.
- Tích hợp LiveData.

- Nhược điểm:

- Cần học cú pháp Room mới.

Tổng kết phần Room

- Room đơn giản hóa việc sử dụng SQLite.
- Dễ dàng tích hợp với các thành phần khác trong Android.
- Thích hợp với ứng dụng hiện đại.

Thực hành với Room

- Nội dung:

- Xây dựng ứng dụng quản lý danh sách sản phẩm.
- Các chức năng: Thêm, hiển thị, xóa sản phẩm.

- Hướng dẫn:

- Tạo Entity, DAO, và Database.
- Hiển thị danh sách sản phẩm với RecyclerView.

Xây dựng giao diện thực hành

- Chi tiết giao diện:

- EditText: Nhập tên sản phẩm, giá.
- Button: Thêm sản phẩm.
- RecyclerView: Hiển thị danh sách sản phẩm.

Tích hợp Room với giao diện

- **Hướng dẫn:**

  - Tạo Adapter cho RecyclerView.

  - Lấy dữ liệu từ Room và hiển thị.

- **Ví dụ:**

```
class ProductAdapter(private val products: List<Product>) : RecyclerView.Adapter<ProductViewHolder>() {
    override fun onBindViewHolder(holder: ProductViewHolder, position: Int) {
        holder.bind(products[position])
    }
}
```

Xóa sản phẩm trong Room

- Hướng dẫn:

- Thêm hàm xóa trong DAO:

```
@Delete
fun deleteProduct(product: Product)
```

- Tích hợp sự kiện xóa với giao diện.

Tổng hợp Room với LiveData

- Hướng dẫn:

- Tự động cập nhật RecyclerView khi dữ liệu thay đổi.

- Ví dụ:

```
viewModel.getProducts().observe(this) { products ->
    adapter.submitList(products)
}
```

So sánh SharedPreferences, SQLite và Room

- **So sánh:**
  - **SharedPreferences:**
    - Lưu dữ liệu khóa-giá trị nhỏ.
    - Dễ triển khai.
  - **SQLite:**
    - Truy vấn SQL đầy đủ.
    - Quản lý dữ liệu lớn.
  - **Room:**
    - Hiện đại, tích hợp LiveData.
    - Dễ sử dụng.

Tổng kết

- Các phương pháp lưu trữ dữ liệu trên Android:

- SharedPreferences: Khóa-giá trị nhỏ.
- SQLite: Truy vấn SQL, dữ liệu lớn.
- Room: ORM, tự động hóa nhiều thao tác.

Ứng dụng thực tế

- Trường hợp sử dụng:

- SharedPreferences: Lưu cấu hình, trạng thái người dùng.

- SQLite/Room: Quản lý dữ liệu lớn, quan hệ phức tạp.

Những kiến thức cần nắm

- Hiểu rõ các phương pháp lưu trữ dữ liệu dài hạn.
- Áp dụng SharedPreferences, SQLite và Room trong các ứng dụng.
- Tích hợp LiveData và ViewModel để tối ưu ứng dụng.

Bài tập thực hành

Bài 1: Tạo cơ sở dữ liệu và bảng

Sử dụng SQLiteOpenHelper để tạo bảng Users

Cột: id, name, age

Bài 2: Chèn & truy vấn dữ liệu

Chèn dữ liệu vào bảng Users

Truy vấn dữ liệu và hiển thị trong Logcat

Bài 3: Nâng cấp cơ sở dữ liệuTăng version từ 1 lên 2

Thêm cột email vào bảng Users

Kiểm tra nâng cấp bằng cách chạy ứng dụng trên thiết bị thực/giả lập mất.

Câu hỏi thảo luận

- Câu hỏi:

1. Điểm khác biệt giữa SQLite và Room?
2. Khi nào nên dùng SharedPreferences?
3. Hãy viết ví dụ sử dụng Room với LiveData.

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Thank you!

THÁI NGUYÊN, THÁNG 10 NĂM 2024